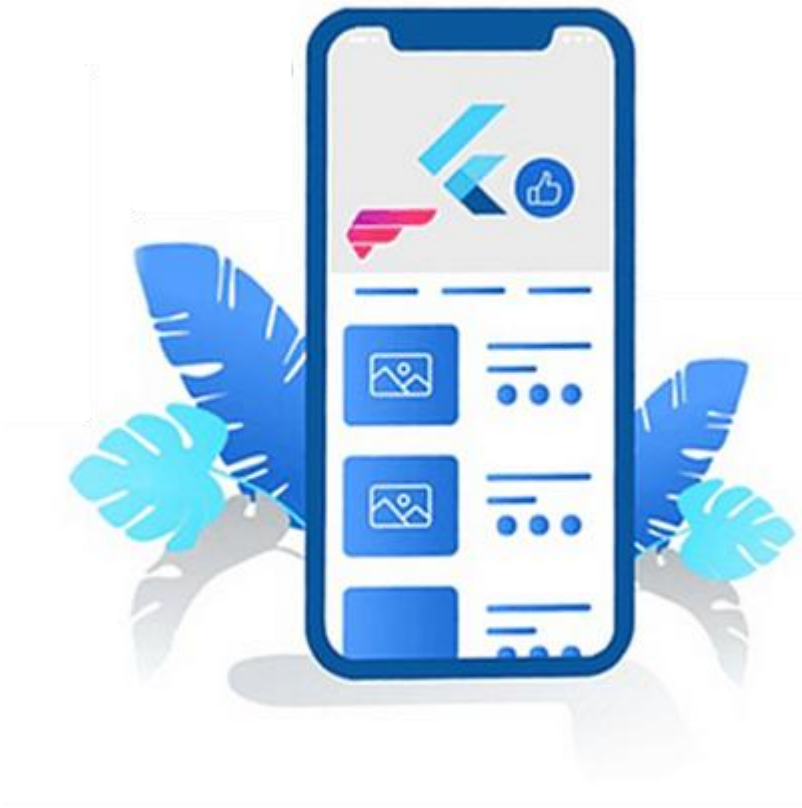
QUIZ

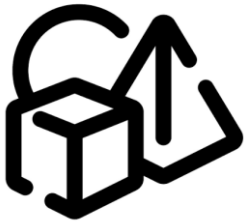
On CADT LMS

WHAT HAVE WE LEARNT ?
SO FAR…



EXPLAIN

Define and **use** classes in Dart



DEFINE CLASS

All classes *except Null* descend from **Object**.

class →

```
class House {
  int rooms;
```

Class Attribute
By default public…
Can be modified outside the class!

Constructor →

```
House(this.rooms);
```

Class Method →

```
  bool isBig() {
   return this.rooms > 5;
  }
}
```

*USE CLASS - **INSTANCIATION***

Create a house With 2 rooms

Create a house With 10 rooms

```
main() {
   House frenchHouse = House(2);

   House khmerHouse = House(10);
   khmerHouse.rooms = 5;

   print(khmerHouse.isBig());
}
```

We can change x attribute here … as it s public

We invoke a method on an object

EXPLAIN

3 ways to manage class **constructors**



```
Point(int x, int y) {
                this.x = x;
                this.y = y;
}
```

Constructor body

Initializer list

```
Point(int x, int y): this.x=x,  this.y=y;
```

```
Point(this.x, this.y);
```

Short form constructor !

EXPLAIN

Getters

A way to provided computed data



CLASSIC GETTER - **FUNCTION SYNTAX**

```
int getWidth() => bottomRight._x - topLeft._x;
```

```
myRectangle.getWith()
```

GETTER WITH **GET KEYWORD**

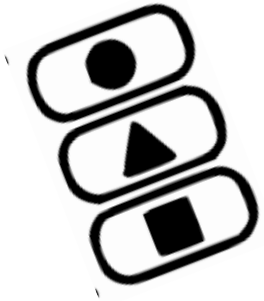
```
int get width => bottomRight._x - topLeft._x;
```

```
myRectangle.width
```

Width is viewed as a new attribute

EXPLAIN

Enumerations *(enums)*



A type of class accepting only a fixed set of class instance

```
enum Color { blue, red, green, yellow }
```

```
Color myColor = Color.blue;
```

EXPLAIN

Make a class **immutable**



MUTABLE

```
class Point {
  int x;
  int y;

  Point(this.x, this.y);

  void translate(int dx, int dy) {
    x+=dx;
    y+-dy;
   }
}
```

IMMUTABLE

```
class Point {
  final int x;
  final int y;

  Point(this._x, this._y);


   Point translate(int dx, int dy) {
    return Point(x + dx, y + dy);
   }
}
```

1- Make attributes **final**

2 – If change requested return a **new object**

I EXPLAIN

Named Constructor



A named constructor is a type of constructor that lets you create an object in different ways

```
ClassName.constructorName(parameters) {
  // constructor body
}
```

```
class Student {

  String name;
  int age;

  // 1 - Default constructor
  Student(this.name, this.age);

 // 2 - Named constructor
  Student.guest() {
    name = "Guest Student";
    age = 0;
  }

  // 3 - Another named constructor
  Student.fromMap(Map<String, dynamic>
data) {
    name = data['name'];
    age = data['age'];
  }
}
```

I EXPLAIN

Operator overloading

*Operator overloading allows you to define the **behavior of some operators** (ex +, -)*

```
class Point {
  int x;
  int y;

  Point(this.x, this.y);

  // Overload the + operator
  Point operator +(Point other) {
    return Point(x + other.x, y + other.y);
  }

  @override
  String toString() => "($x, $y)";
}

void main() {
  var p1 = Point(2, 3);
  var p2 = Point(4, 5);
  var result = p1 + p2;
  print(result);
}
```



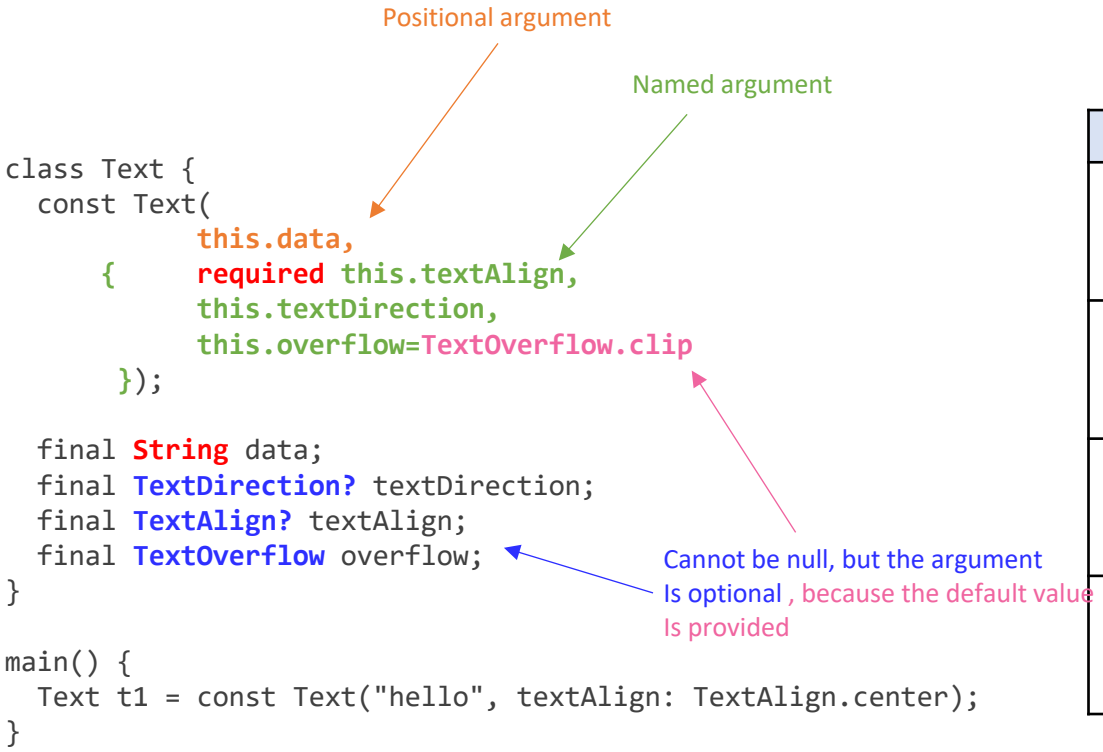
if point does not extend anything, why toString() have @override ?



What will this code output?

EXPLAIN

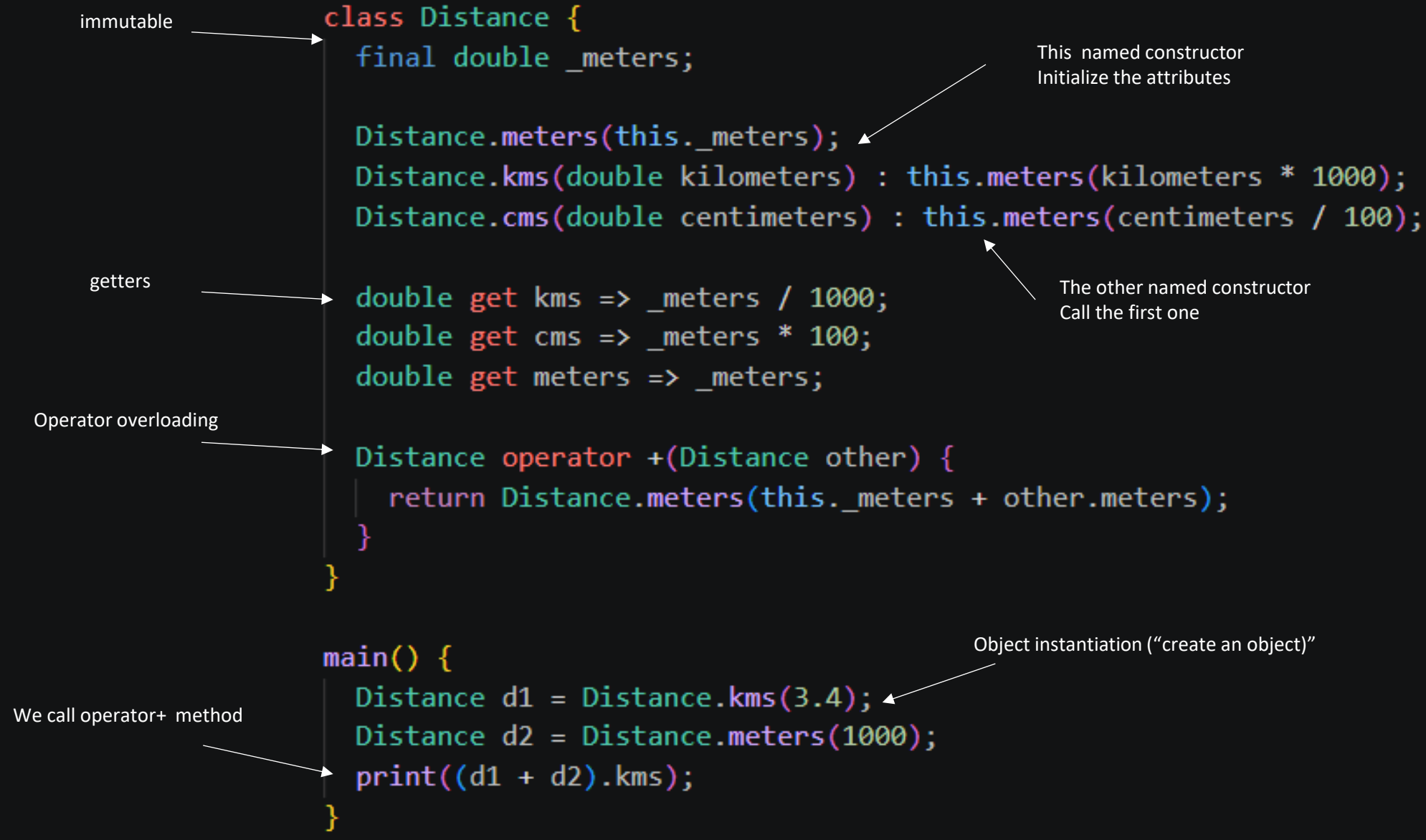
Positional, named, optional, mandatory, default **arguments**



Argument	Positional / Named	Mandatory / Optional	Default value?
data	positional	mandatory	no
textAlign	named	optional	no
textDirection	named	optional	no
overflow	named	optional	default values

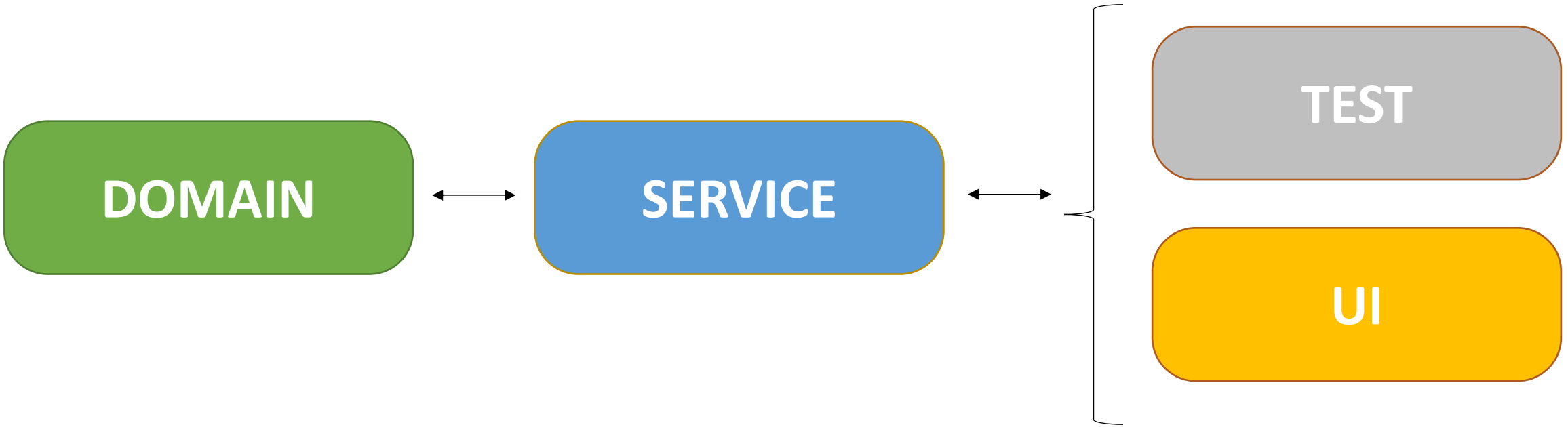


EXAMPLE


immutable
class Distance {
final double _meters;
This named constructor
Initialize the attributes
Distance.meters(this._meters);
Distance.kms(double kilometers) : this.meters(kilometers * 1000);
Distance.cms(double centimeters) : this.meters(centimeters / 100);
getters
The other named constructor
Call the first one
double get kms => _meters / 1000;
double get cms => _meters * 100;
double get meters => _meters;
Operator overloading
Distance operator +(Distance other) {
return Distance.meters(this._meters + other.meters);
}
}
main() {
Object instantiation (“create an object)”
Distance d1 = Distance.kms(3.4);
We call operator+ method
Distance d2 = Distance.meters(1000);
print((d1 + d2).kms);
}

MOBILE DEVELOPMENT

W3 – Layered Architecture



CADT IDT



Session Objectives



- ✓ Represent well **a one-to-many relationship** between objects *(A has many B)*
  - COMPOSITION — NESTED OBJECTS
  - AGGREGATION — OBJECTS REFERENCES
  - AGGREGATION — IDs REFERENCES

- ✓ Structure and document your project architecture **around 3 layers**
  - MODEL layer
  - SERVICE layer
  - TEST layer

- ✓ Implement your **business functional requirements** using service **PRE/POST conditions and unit tests**

A **field** has many **sheep**



3 way to represent the relationship “**A has many B**”

OPTION 1

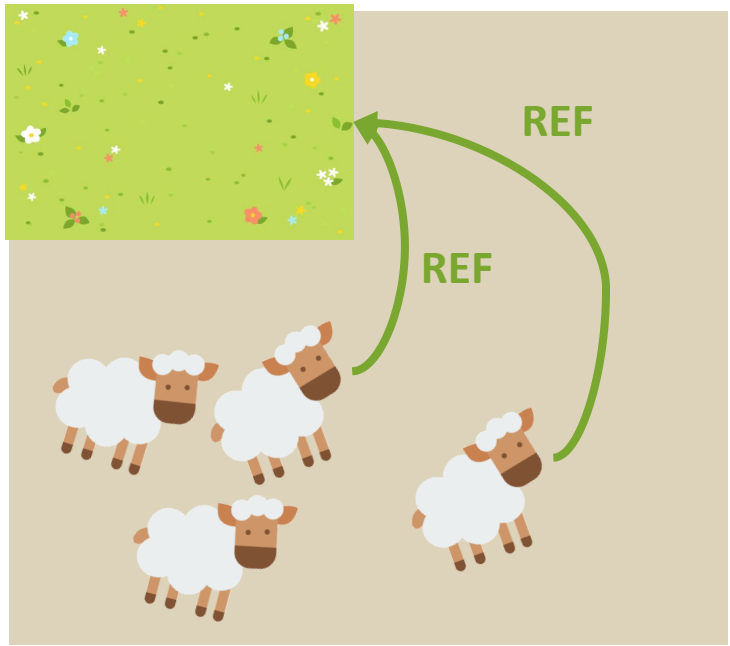
COMPOSITION — NESTED OBJECTS



- ✓ A contains a List<B> of objects that it owns.
- ✓ When A is destroyed, its B objects are also destroyed.

OPTION 2

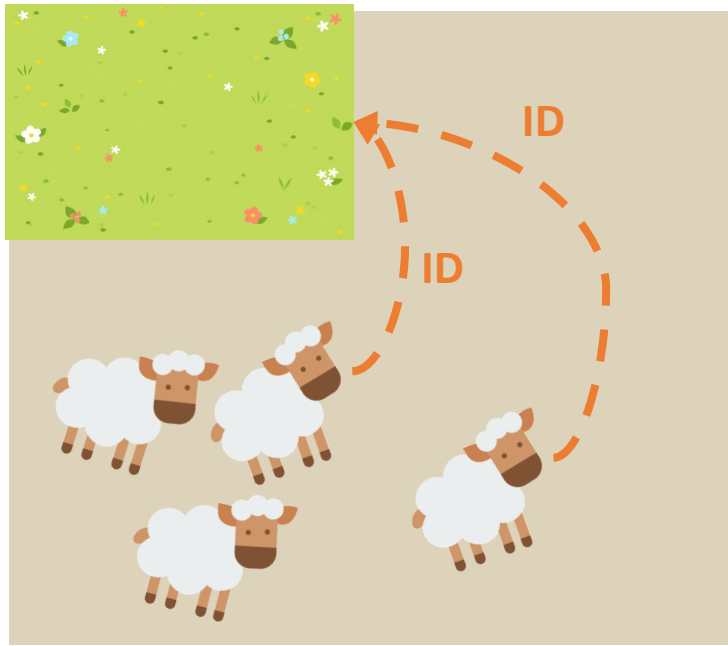
AGGREGATION — OBJECTS REFERENCES



- ✓ A contains a List<B> of references to existing B objects.
- ✓ The B objects exist independently of A.
- ✓ When A is destroyed, the B objects can continue to exist.

OPTION 3

AGGREGATION — IDs REFERENCES



- ✓ A contains a List<String> of ids to existing B objects.
- ✓ The actual B objects are retrieved using their IDs.
- ✓ When A is destroyed, the B objects can continue to exist.

Link the 3 codes to their approach:

COMPOSITION — NESTED OBJECTS

AGGREGATION — IDs REFERENCES

AGGREGATION — OBJECTS REFERENCES

CODE 1

```
class Sheep {
  Sheep(this.id, this.name);
  final String id, name;
}

class Field {
  final List<String> sheepIds = [];

  void addSheep(String sheepId) {
    sheepIds.add(sheepId);
  }
}



void main() {
  List<Sheep> sheeps = [
    Sheep('s0', 'Dolly'),
    Sheep('s1', 'Shaun'),
    Sheep('s2', 'Molly'),
    Sheep('s3', 'Billy'),
  ];

  var field1 = Field([sheeps[0].id, sheeps[3].id]);
  var field2 = Field([sheeps[1].id, sheeps[2].id]);
}
```

CODE 2

```
class Sheep {
  Sheep(this.id, this.name);
  final String id, name;
}

class Field {
  final List<Sheep> sheep = [];

  void addSheep(
           String id,
           String name) {
    sheep.add(Sheep(id, name));
  }
}



void main() {
  Field field = Field();

  field.addSheep(('s0', 'Dolly');
  field.addSheep(('s1', 'Shaun');
}
```

CODE 3

```
class Sheep {
 Sheep(this.id, this.name);
  final String id, name;
}

class Field {
  final List<Sheep> sheep = [];

  void addSheep(Sheep sheep) {
    sheep.add(sheep);
  }
}



void main() {
  List<Sheep> sheeps = [
    Sheep('s0', 'Dolly'),
    Sheep('s1', 'Shaun'),
    Sheep('s2', 'Molly'),
    Sheep('s3', 'Billy'),
];

 var field1 = Field([sheeps[0], sheeps[3]]);
 var field1 = Field([sheeps[1], sheeps[2]]);
}
```

Link the 3 codes to their approach:

COMPOSITION — NESTED OBJECTS

AGGREGATION — IDs REFERENCES

AGGREGATION — OBJECTS REFERENCES

CODE 1

```
class Sheep {
  Sheep(this.id, this.name);
  final String id, name;
}

class Field {
  final List<String> sheepIds = [];

  void addSheep(String sheepId) {
    sheepIds.add(sheepId);
  }
}



void main() {
  List<Sheep> sheeps = [
    Sheep('s0', 'Dolly'),
    Sheep('s1', 'Shaun'),
    Sheep('s2', 'Molly'),
    Sheep('s3', 'Billy'),
  ];

  var field1 = Field([sheeps[0].id, sheeps[3].id]);
  var field2 = Field([sheeps[1].id, sheeps[2].id]);
}
```

The field keeps only the **sheep IDs**

CODE 2

```
class Sheep {
  Sheep(this.id, this.name);
  final String id, name;
}

class Field {
  final List<Sheep> sheep = [];

  void addSheep(
           String id,
           String name) {
    sheep.add(Sheep(id, name));
  }
}



void main() {
  Field field = Field();

  field.addSheep(('s0', 'Dolly');
  field.addSheep(('s1', 'Shaun');
}
```

The field **manages its own sheep**

CODE 3

```
class Sheep {
 Sheep(this.id, this.name);
  final String id, name;
}

class Field {
  final List<Sheep> sheep = [];

  void addSheep(Sheep sheep) {
    sheep.add(sheep);
  }
}



void main() {
  List<Sheep> sheeps = [
    Sheep('s0', 'Dolly'),
    Sheep('s1', 'Shaun'),
    Sheep('s2', 'Molly'),
    Sheep('s3', 'Billy'),
];

 var field1 = Field([sheeps[0], sheeps[3]]);
 var field1 = Field([sheeps[1], sheeps[2]]);
}
```

The field keeps **references to sheep** (pointer)

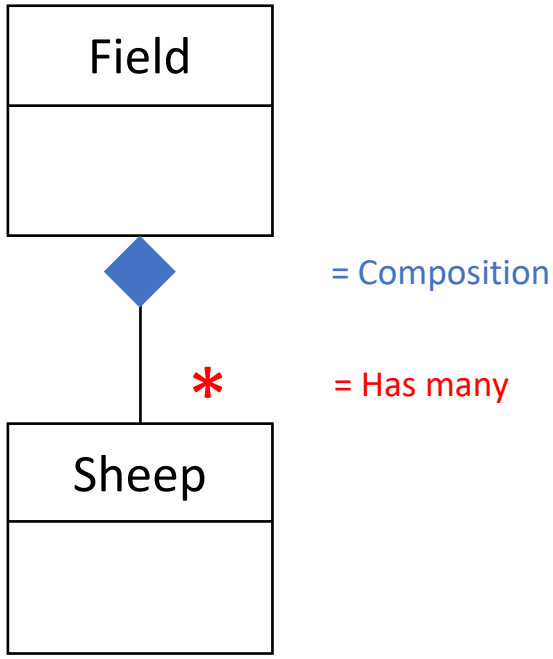
A **field** has many **sheep**



3 way to represent the relationship “**A has many B**” in UML

OPTION 1

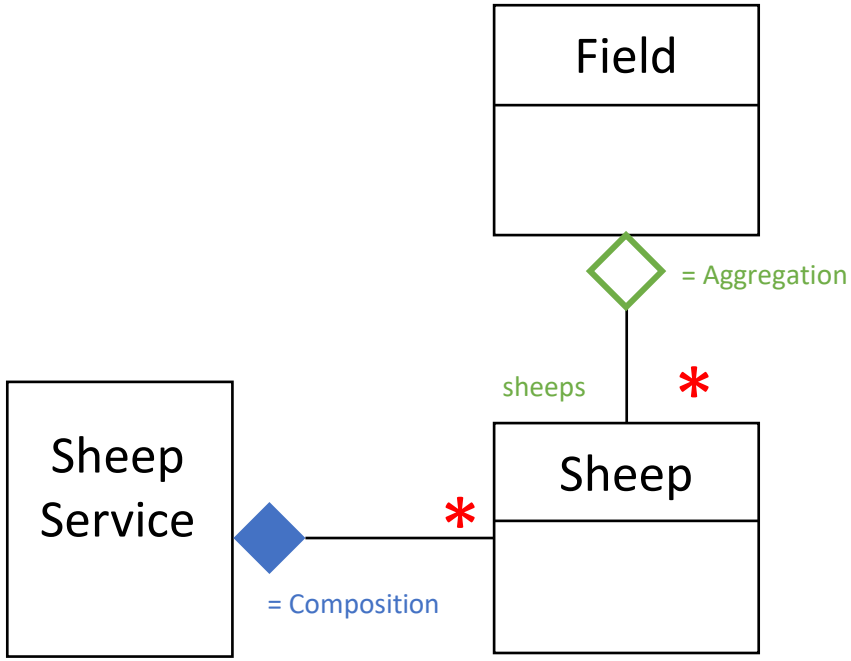
COMPOSITION — NESTED OBJECTS



- ✓ **Field** contains a List<**Sheep**> of objects that it owns.
- ✓ When **Field** is destroyed, its **sheep** objects are also destroyed.

OPTION 2

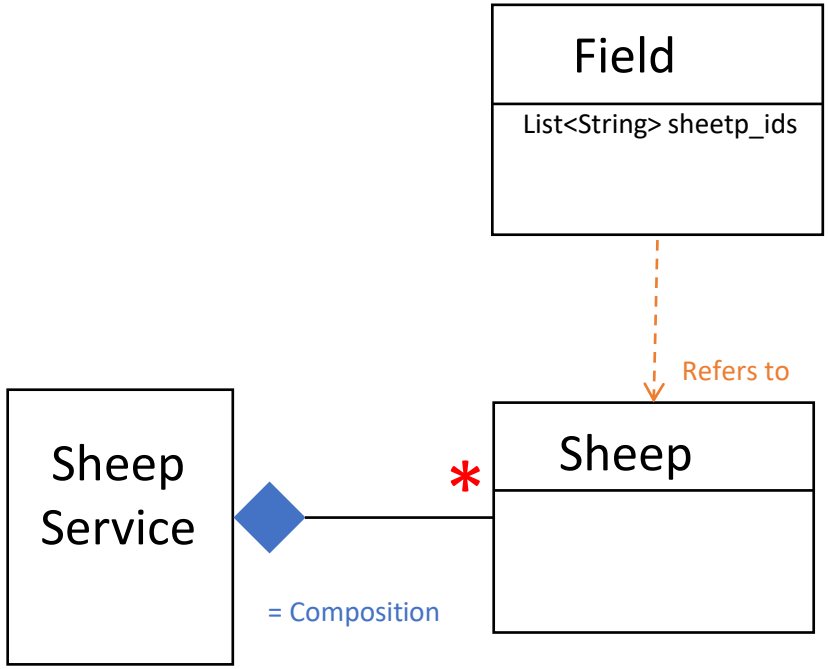
AGGREGATION — OBJECTS REFERENCES



- ✓ **Field** contains a List<**Sheep**> of references to Sheeps
- ✓ When **Field** is destroyed, its **sheep** objects are NOT destroyed.

OPTION 3

AGGREGATION — IDs REFERENCES



- ✓ When the **Field** contain a list of **Sheet** ID, we simply **indicate a dependency** (dashed line)


Layered
ARCHITECTURE

EXPLAIN

Layered Architecture

We organize a project into separate **layers**, each with a distinct **responsibility**

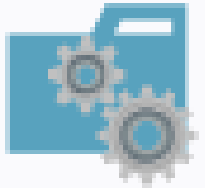
LAYER	RESPONSABILITY
MODEL	✓ Contains the classes representing the **business data**
SERVICE	✓ Owns the **business data** *(for now !)* ✓ Provides **functions** that other parts of the application can use.
TEST	✓ **Verifies** that the **service functions** behaves correctly.

⚠️ This is a simplified version – We will add other layer along term 1 and term 2..

Each layer will be a **folder** of our application project



model



service



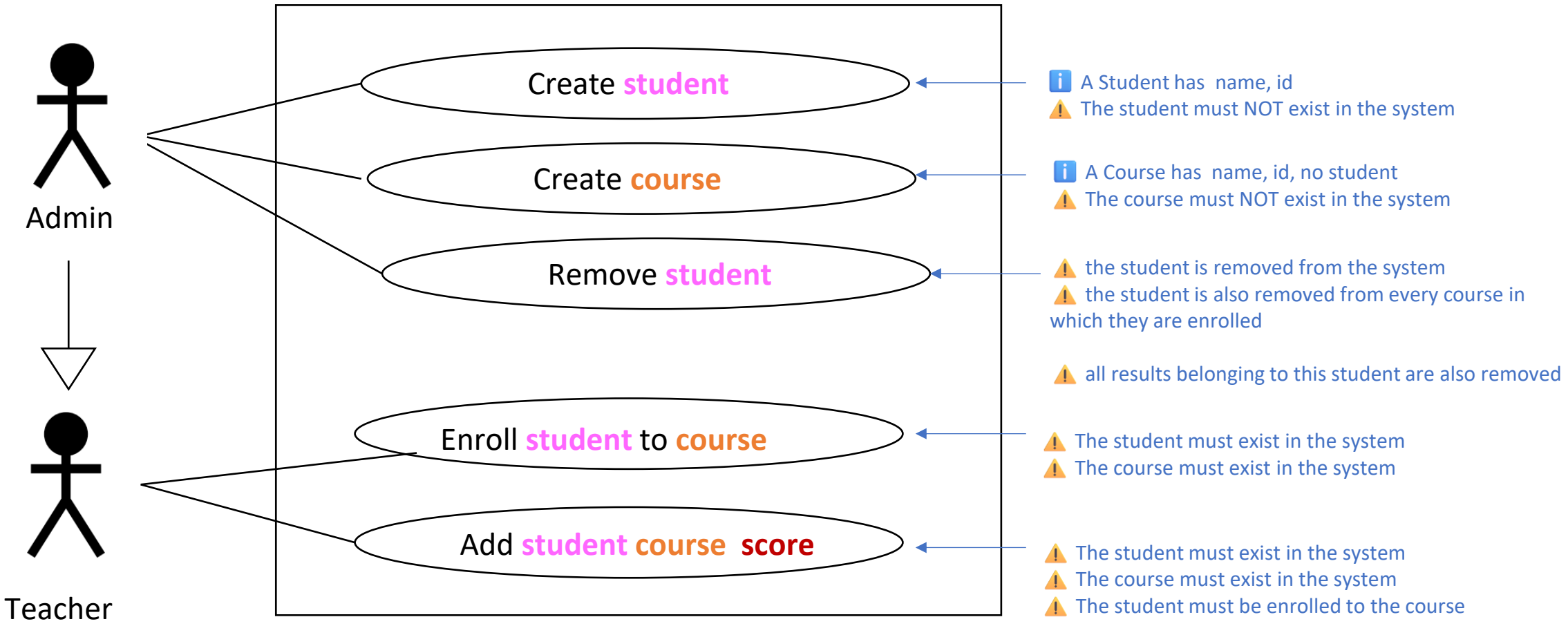
test

EXAMPLE – From 0 to HERO



Let’s design a Learning management System

Our system can be represented with the following **use cases:**



MODEL

The model layer contains the classes representing the **business data**



What classes/enum do I need ?

- Student
- Course,
- CourseResult



What classes attributes do I need ?

Student

String id
String name

Course

String id
String name
List<?> enrolledStudents

CourseResult

String courseId
String studentId
double score
String comment

MODEL

There are several possible ways to handle **courses-students** relationship

OPTION 1

COMPOSITION — *Students are nested in course*



```
Course
  └── List<Student>
```

This creates a potential **data duplication**:

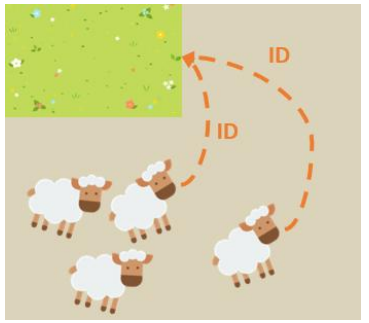
```
Course A
  └── Student S01

Course B
  └── Student S01
```

As a student can be enrolled to many course, this approach will duplicate student data

OPTION 2

AGGREGATION —*Students are referenced by courses*




```
StudentsService
  └── List<Student>

Course
  └── List<String> studentIds
```

Now Course just refers to a student (ID or Object reference)

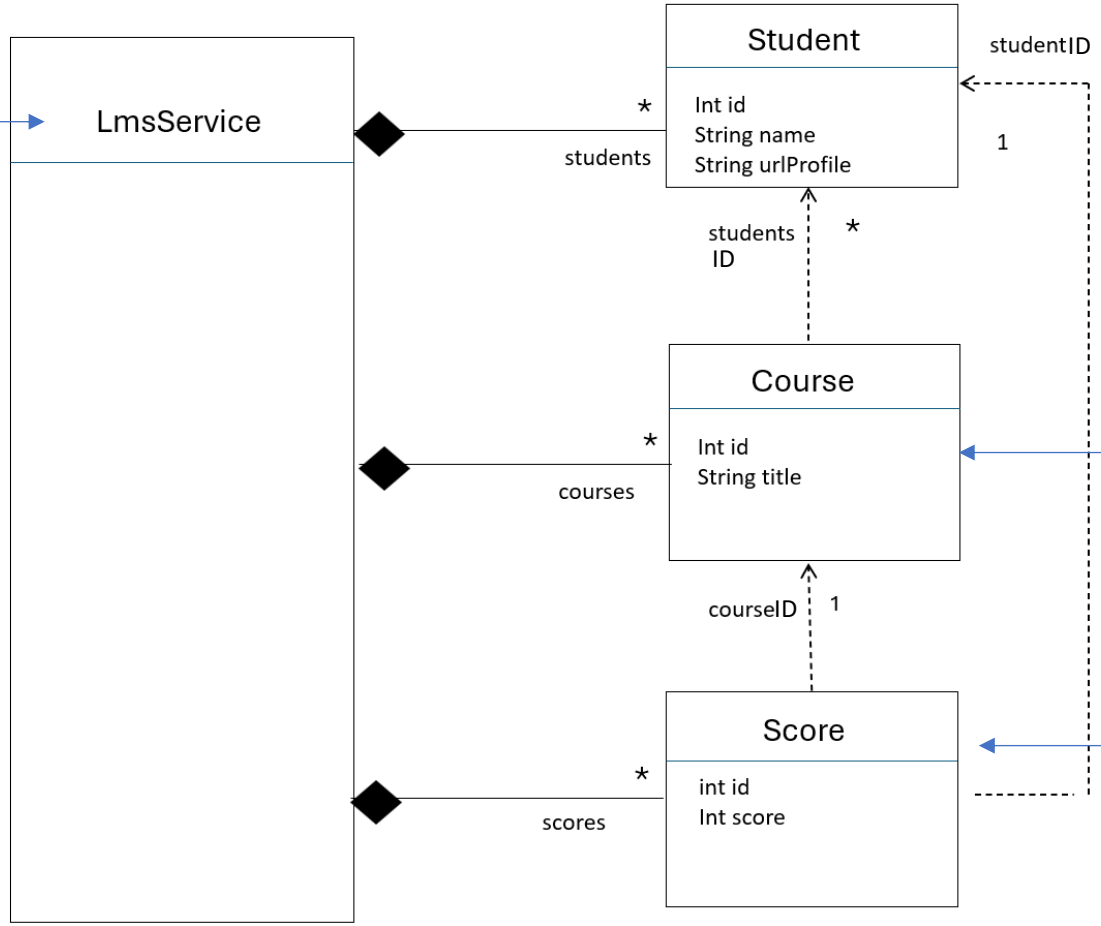
With this approach, a student can be **enrolled to many courses**, without data duplication

MODEL

We finally came up the following model

The service **gathers the app data** :

```
Students
Course
Course results
```



⚠️ This is a simplified version – We will add other layer along term 1 and term 2..

SERVICE

The service is responsible for **maintaining the consistency** of the system.

For example, the following situation should never happen

```
Student S01 does not exist in the system
```

BUT

```
Course C01 contains student S01
```

INCONSISTENCY

💡 This is one of the reasons for having a Service layer.

TEST

SEE MORE ABOUT DART TEST

The **Test layer** creates a **service** and tests its public **operations**.

A typical test follows this pattern:

```
Create the Service
        ↓
Add data
        ↓
Call LMS  service operation
        ↓
Check result
```

DEMO

Let's DO it

MODEL

Create a class **Student** and **Course**

- ℹ️ A Student has name, id
- ℹ️ A course has name, id and list of enrolled students

SERVICE

LMS Service : Create an operation to **add a student**

- ⚠️ The student must NOT exist in the system

LMS Service : Create an operation to **remove a student**

- ⚠️ The student must exist in the system

TEST

LMS Test : Create a test to add a student

LMS Test : Create a test to remove a student

💡 `flutter pub add test`

MODEL

```
class Student {
  final String id;
  final String name;

  Student({required this.id, required this.name});
}
```

```
class Course {
  final String id;
  final String name;
  final List<String> studentIds = [];

  Course({required this.id, required this.name});
}
```

SERVICE

```dart
void addStudent({required String studentId, required String studentName}) {

  // ! Student shall not exist already
  Student? student = _findStudentOrNull(studentId);
  if (student != null) {
    throw Exception('Student $student already exists');
  }


  students.add(Student(id: studentId, name: studentName));
}
```

```dart
void removeStudent({required String studentId}) {
  Student? student = _findStudentOrNull(studentId);
  if (student == null) {
    throw Exception('Student $studentId not found');
  }
  // 1- Remove student from the students list
  students.removeWhere((student) => student.id == studentId);

}
```

TEST

💡 flutter pub add test

```dart
test('creates a student', () {
  final lms = LmsSystem();


  // Create a student
  lms.addStudent(studentId: 'S1', studentName: "ronan");

  expect(lms.students.first.name, 'ronan');


  // Remove the student
  lms.removeStudent(studentId: 'S1');
  expect(lms.students.length, 0);
});
```

To wrap up



Single

RESPONSABILITY

A class should do **one thing** and do it **well**



What do you know now



- ✓ Represent well **a one-to-many relationship** between objects *(A has many B)*
  - COMPOSITION — NESTED OBJECTS
  - AGGREGATION — OBJECTS REFERENCES
  - AGGREGATION — IDs REFERENCES
- ✓ Structure and document your project architecture **around 3 layers**
  - MODEL layer
  - SERVICE layer
  - TEST layer
- ✓ Implement your **business functional requirements** using service **PRE/POST conditions and unit tests**